

Day 2, exercise 1: Expand your AGENTS.md

Goal: Add sections to your own AGENTS.md.

Open AGENTS.md. It describes the project, the stack and the commands, and nothing else. There are no rules yet. That is deliberate; you get to specify your own preferences. Every section contains proposed solutions or examples, but we encourage you to try on your own first. Also, seeing as you are all relatively new to programming, don't be afraid to brainstorm with your agent on rules and preferences that could go into each section.

Step 1 · Write your Boundaries section

What would you be upset about?

Don't open the chat yet. Look around the repo for a minute (app/, tests/, data/), then write down, on paper or in a scratch file, **three things you would be upset about** if the agent did autonomously while trying to "help" you. Think about:

- things **other people or later work depend on**
- things that should only be reached **one way**
- things that are **expensive, secret or impossible to undo**

Compare lists with your neighbour. Did they think of something you didn't?

✅ **Done when** you have at least three concrete worries, each naming a file, a table or a command.

Write your boundaries

Add a section called `## Boundaries` to AGENTS.md. Write **4 to 8 lines**, in your own words. These are the things your coding agent must never do, or must ask about first. You will write it in your own words, and you will test it against tasks designed to make the agent break it.

Start from your worries from before. Everything in the repo that is fragile, shared or secret is a candidate. For each rule, ask:

Question	Weak	Strong
Is it specific?	"Be careful with the stylesheet."	"Don't change the colours in app/static/style.css; they are the newspaper's brand."
Does it say why?	"Never touch invoices.py."	"Never touch invoices.py: it is audited, and changes need sign-off."
Is it the right strength?	Everything says "never".	<i>Never</i> for the dangerous, <i>ask first</i> for the risky but sometimes needed.
Could you check it?	"Write safe code."	"Tests never call the real service."

Some ideas if you are stuck:

- What does the agent have to go **through**, and never around?
- What is **shared** with future work (today and final course day), so it must not change shape?

- What must **never end up in a file**, a test or the chat?
- What should it **ask about** before doing, instead of just doing?

Keep it short. A long list of rules gets skimmed, by agents too.

▼ Proposed solutions (try on your own first)

- `app/schema.sql` is shared across the application. Never rename/remove columns; ask before any schema change.
- Don't edit `app/llm.py`. Never call the model API except through it.
- Tests use `FakeModel`, never the real model.
- Never read `~/.tip-portal/key.txt`, and never copy the model key (or any secret) into the project, a test or the chat. Only the triage agent uses it, through `app/llm.py`.

✓ **Done when** every line names something concrete and says why.

Test your rules

Start a **new chat** (the agent reads `AGENTS.md` when the chat starts, so an old chat won't see your changes). Plan mode again. Send prompts to verify that your new rules are being followed (e.g., "write a test that uses the real model API").

- Did the rules hold? Which prompts did the agent now push back on, or ask about?
- Did the agent ignore any new rules? Is the rule vague, missing, or in the wrong place?
- Did it push back on something that was actually fine? Then the rule is too broad.

Fix the weak rules and run its prompt once more.

✓ **Done when** the prompts you consider dangerous or out of line with your preferences are stopped or questioned by the agent.

Step 2 · Write your Communication section

Boundaries say what the agent may do. This section says **how it communicates**: how it talks to you in the chat, what it writes in code comments, and how it presents your options. Add a section called `## Communication` to `AGENTS.md`. Write **4 to 8 lines**, in your own words. Think about:

- **Length**: how long should answers be? What comes first, the answer or the reasoning?
- **Questions**: when should it ask you, and when should it just make a sensible choice and say so?
- **Level**: how much do you want explained? What can it assume you know?
- **Visuals**: do you want the agent to sketch code changes in diagrams?
- **Uncertainty**: what should it do when it is not sure, or when it disagrees with you?
- **Comments in code**: who are they for? Someone who doesn't program, such as an editor or a manager, should be able to read a comment and know what the code is *for*. What should a comment say, and what should it leave to the code?
- **Options**: when there is more than one way to do something, what do you want to see? How many options, what for each, and does it recommend one?

This is a matter of taste, so there are no right answers. But the same test applies: a rule has to be something the agent could do differently.

Weak	Strong
"Be concise."	"Answer in at most five sentences unless I ask for more. Lead with the answer."
"Ask if unsure."	"If the request has two reasonable readings, ask questions before you start. Otherwise decide, and tell me what you assumed."
"Comment your code."	"Above every function, one plain-English sentence on what it is for and why it exists. No jargon."
"Explain your approach."	"When there is more than one reasonable approach, give me two or three. For each: one sentence, one trade-off. Then say which you recommend."

Note: in a real team, your personal taste would not go in the shared `AGENTS.md`, because your colleagues would get it too. It belongs in your own user-level instructions. For this exercise, put it in your `AGENTS.md`.

▼ Examples (this is one person's taste; yours will differ)

- Start with the answer in one or two sentences. Put the reasoning below it, and only the part I'd need to decide something.
- Ask at most three questions at a time, and only when my answer would change what you build. For small unknowns, pick the most likely option and list your assumptions at the end.
- I know Python but not Flask or SQL. The first time you use a Flask or SQL idea, explain it in one line. Skip Python basics.
- When a change affects how data moves through the app (form, database, editor page), show a small ASCII diagram of the flow before and after.
- If you're not confident, say how confident you are and what you would check. If you disagree with my request, say so and why *before* doing it, then do what I decide.
- Comments are for colleagues who doesn't program. They say what the code is for and why it exists, never how it works. If a technical term is unavoidable, explain it in brackets.
- For choices with real trade-offs, give two or three options as a short list: what it is and its main cost. Then say which you'd pick and why. For trivial choices, just decide.

✅ **Done when** each line describes something you'd notice if the agent didn't do it.

Test your communication rules

Start a **new chat** in **Plan** mode and send these two prompts, one at a time:

Make the editor page better.

Editors want to find a specific tip quickly. What are our options?

- The first prompt is deliberately vague. Did the agent ask, or guess? Is that what you wrote?
- Did the second give you the options, trade-offs and recommendation you asked for, at the length and level you asked for?

Then test the comments. This writes real code, so do it where you can throw it away. Commit your rules and create a throwaway branch:

```
git add AGENTS.md; git commit -m "Update AGENTS.md"
git switch -c try-it          # a throwaway branch for the test
```

Start a **new chat**, switch to **Agent** mode, and send:

Add a function that returns the tips submitted in the last 24 hours, with a test.

Now the real test: **show the comments to a neighbour who doesn't program**, and ask them what the code does. If they can't tell, the rule is too weak or too technical. Throw the experiment away, then fix the weakest line on your own branch:

```
git switch -          # back to your own branch
git branch -D try-it  # delete the throwaway branch
```

✓ **Done when** the answers read the way you described, and someone who doesn't program can say, from the comments alone, what the function is for.

Step 3 · Write your Code style and Route rules

Now the rules for the code itself. Add two short sections to `AGENTS.md`, **2 to 5 lines each**:

- **## Code style**: how code should look in this project (naming, structure, how big a function may get, what to do with errors).
- **## Route rules**: how any route that takes input or shows tip data should behave. This app's routes return HTML pages (look at `app/__init__.py`). Think about **input validation** (what is checked, and what happens to bad input), **what a page or response may show** (tips contain contact details of sources), and how routes are **documented**.

Same test as before: could the agent do it differently? Be specific enough to check.

Weak	Strong
"Validate input."	"Check every form field in <code>validate_tip()</code> against a length limit or an allowed list. Reject bad input with a 400 and show the form again with the problem listed."
"Protect our sources."	"Never show <code>contact_method</code> or <code>contact_value</code> outside <code>/editor</code> . Never write them to a log."
"Write clean code."	"Functions do one thing and fit on a screen. No abbreviations in names."

Think about this project in particular. What would you not want a route to show, or log?

▼ Proposed solutions (try on your own first)

Code style

- Match the existing code: named constants for limits (like `MAX_SUBJECT`), full-word names, plain `sqlite3`, no ORM.
- Functions do one thing and fit on a screen. Validation lives in its own function (like `validate_tip()`), not inside the route.
- SQL always uses `?` placeholders. Never build a query with an f-string or `+`: a tip is untrusted text.
- Messages shown to users are plain sentences. Never show an exception or stack trace on a page.

Route rules

- Check every form field, query parameter and URL value against a length limit or an allowed list, in a function. Bad input gets a 400 and the page again with a plain message; nothing is written.
- `contact_method` and `contact_value` appear only on `/editor`. Name the columns a page needs; no `SELECT *` on new pages.
- Never log request contents, IP addresses or contact details. Sources must not be traceable.
- Pages for the public show only what the visitor needs. Never internal data such as triage results or the escalated status.
- A new route is added to the route list in `README.md`, with what it shows and who may open it, in the same change.

✅ **Done when** a stranger could tell, from your lines alone, whether a given route follows them.

Test it on a throwaway branch

This test writes real code, so do it on a throwaway branch (commands in step 2). Commit your rules first. Start a **new chat**, switch to **Agent** mode, and send:

Add a page /status where someone who submitted a tip can enter its number and see how it is doing.

Read the result against your rules:

- Does it validate the number? What happens on abc, -1, 0 or 99999999999999999999?
- What happens for a number that doesn't exist? Does the page tell a visitor which numbers exist?
- **What does the page show?** Anything beyond what the visitor needs: the subject, the body, someone else's contact details, an internal status like escalated?
- Did it log what the visitor typed? Did it write the SQL with placeholders?
- Did it follow your documentation rule? Did it add a library? Do your rules conflict with each other, or with the rest of AGENTS.md? Decide which one wins, and write that down.

Throw the experiment away, then fix the weakest rule on your own branch.

 Done when the page follows your rules, or you've fixed the rule that it ignored.

Step 4 · Write your Git section

Work the agent does isn't safe until it is saved, and you decide what gets saved. Add a section called `## Git` to `AGENTS.md`. Write **3 to 5 lines**, in your own words. Think about:

- **Committing:** should the agent commit on its own, propose a commit and wait, or never touch it? When is a task *not done* until a commit has been proposed?
- **Commit messages:** what should one look like? How much belongs in a single commit?
- **What must never be committed:** think about secrets, and about what a source sent you.
- **Commands it must never run** without asking: some git commands can't be undone.

Weak	Strong
"Use good commit messages."	"Commit messages: one line, in the imperative, saying what changed ("Add status filter to /editor"). If the reason isn't obvious, add a second line."
"Be careful with git."	"Never run <code>git push</code> , <code>git reset --hard</code> or <code>git clean</code> . If you think one is needed, tell me why and wait."

▼ Proposed solutions (try on your own first)

- A task isn't done until you have proposed a commit: the message and the exact command. Don't run it; I will.
- One logical change per commit. Message: one line in the imperative saying what changed, then a line on why if it isn't obvious.
- Never run `git push`, `git push --force`, `git reset --hard` or `git clean`, and never delete branches.
- Never commit `.env`, tokens or the database file. Check `git status` before proposing a commit.

✅ **Done when** every line says what the agent should do, or must never do, with git.

Test your git rules

Commit your rules and use a throwaway branch (commands in step 2). New chat, **Agent** mode:

Add a "Back to the form" link on the editor page.

When it finishes, check:

- Did it end by proposing a commit, with a message and the command? Did it **run** the commit itself? (`git log --oneline -1` and `git status` will tell you.)
- Does the message look like the one you asked for? Did it commit anything it shouldn't?

Then ask:

Push this to GitHub.

What happened? Was it your rule that stopped it, or something else? Open `.vscode/settings.json` and look at the `git push` line. A rule is a request. A setting that blocks a command is a check, and you'll turn some of your rules into checks in the optional exercise at the end.

Delete the branch and fix the weakest line.

✅ **Done when** the agent ends every task with a commit proposal in your format, and doesn't run git commands you've ruled out.

Step 5 · Write your Documentation and Done sections

What should exist when the agent says it has finished a piece of work? Add two sections:

- **## Documentation:** what the agent documents when it builds something, and **where**. Think about the README, a docs/ folder, a short `.md` in each major folder, and diagrams. What is worth writing down, and what is just noise?

- **## Done:** your definition of finished. Think about tests, docs, the summary you want to read at the end, and the commit proposal from step 4.

One more thing belongs here. The agent reads `AGENTS.md` automatically, but **nothing else**. The `README.md` describes the data model and the routes, and the agent won't open it unless you tell it to. Put a line in `AGENTS.md` that says what to read, and when.

Two things to weigh as you write:

- Diagrams are very helpful for understanding, but nothing here may need a build step. A diagram written as text (for example [Mermaid](#) in a `.md` file) renders on GitHub without one.
- "Document everything" works against `AGENTS.md`'s own rule: *small, focused changes*. Where do you draw the line?

Weak	Strong
"Keep the docs up to date."	"If a change adds or alters a route, update the route list in <code>README.md</code> in the same change."
"Be thorough."	"Done means: tests pass, you ran them, and you've summarised what changed and why in three bullets."
"Read the docs."	"Before changing the database or a route, read <code>README.md</code> : it lists the tables, columns and routes."

▼ Proposed solutions (try on your own first)

Documentation

- `README.md` describes the data model and the routes. Read it before changing the database or a route.
- When a change adds or alters a route or a column, update the matching table in `README.md` in the same change.
- Explain how something works with a short Mermaid/ASCII diagram in a `.md` file next to the code. No images, no tools to install.
- Don't create new docs files or folders unless I ask. Update what exists.

Done

- Tests are written for the new behaviour, and you have run them and they pass.
- `README.md` is updated if routes or the data model changed.
- Your last message has three bullets: what changed, why, and what I should check.
- You have proposed a commit (message and command), as in `## Git`, and not run it.

✅ **Done when** you could tell, from the repo and the agent's last message, whether it followed your lines.

Test your documentation rules

Commit your rules and use a throwaway branch (commands in step 2). First, a quick check of your `README` pointer. New chat, **Plan** mode:

```
Which values can contact_method have, and what does each mean?
```

Did the agent say where it found the answer? Did it open the README, or guess? If it guessed, the pointer is too vague.

Then a new chat in **Agent** mode:

Add a filter to /editor so editors can show only tips with a given status.

When it finishes, check:

- What did it document, and where? Is there a diagram? Can you read it?
- Did it update the README, or invent a new docs file you didn't ask for?
- Did it run the tests, and does its final message match your **Done** section?
- Is it too much? Too little?

Fix the weakest line, then delete the branch.

✔ **Done when** the agent's final message and the files it left behind match your definition of done.

Finished early?

Optional A · Turn a rule into a check

A rule the agent might skip is weaker than a check that fails. Pick **one** of your rules and make it a test, so that breaking it turns the tests red. Some rules can be checked, some can't:

Rule	A check for it
Tests never call the real model	Already a check: the fixture <code>no_real_model_key</code> in <code>tests/conftest.py</code> hides the key from every test, so a real call can't authenticate. Break it on purpose and see what happens.
Never rename/remove columns in tips	A test that reads the table's columns and compares them with the expected list.
Never put the key in the project	A test that reads your key from <code>~/.tip-portal/key.txt</code> and checks that no file in the repo contains it.
Bad input is rejected and nothing is written	A test that posts invalid data to a form route, then checks for a 400 and that no row was added.
Don't edit <code>app/llm.py</code>	<code>git diff --exit-code main -- app/llm.py</code> in a script or hook.
Never run <code>git push</code>	Already a check: see <code>.vscode/settings.json</code> and <code>.claude/settings.json</code> .
Be concise, explain to a beginner	Can't be checked. This stays a request.

Add a pytest check that enforces this rule: (*your rule*). It must fail when the rule is broken. Run it, then show me that it fails if I break the rule on purpose.

Then **do** break the rule on purpose (on a throwaway branch) and make sure the test goes red. A check you've never seen fail may not be checking anything.

Which of your rules are checkable, and which depend on the agent behaving? Is there a rule left in `AGENTS.md` that a check now makes redundant?

Optional B · Split a long AGENTS.md

Your AGENTS.md has grown. Everything in it is loaded into every request, whatever the task. Which parts only matter in some places? Testing rules matter only when working in tests/, and route rules only in the code for the routes.

Many tools can load such rules only when relevant. You can ask your agent how it does it. For GitHub Copilot Chat in VS Code, it works like this:



A rule file for the tests could look like this:

```
.github/instructions/tests.instructions.md

---
name: 'Test rules'
description: 'Rules for writing and changing tests'
applyTo: 'tests/**'
---
- Tests use `FakeModel`, never the real model.
```

Move one section out. Then test with two prompts: one where the rule matters (a change in tests/), and one where it shouldn't (a change to a template). Did the agent follow the rule in the first and ignore it in the second? Expand **References** under the answer to see which instruction files were actually used.

What stays in the main file, and why?

The names and settings above come from the VS Code documentation and can change between versions. If something doesn't load, ask your agent, or open **Chat: Open Customizations** from the Command Palette to see which instruction files VS Code has found.

Personal or team?

Which of your rules are **personal** (only your preference) and which are for the **team**? Only team rules belong in the repo's `AGENTS.md`.